

# **COMP0496 – Programação A**

## **Programação Orientada a Objetos — Parte 2**

Encapsulamento e Métodos Especiais

Prof. Dr. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

Revisão — O Problema do Estado Inválido

Encapsulamento

@property — Acesso Controlado

Métodos Especiais (Dunders)

Prática — Classe Fracao

Resumo

# **Revisão — O Problema do Estado Inválido**

Na Aula 1, criamos classes para o food truck **O Podrão**.

```
1 class Lanche:
2     def __init__(self, nome, preco):
3         self.nome = nome
4         self.preco = preco
5
6 xtudo = Lanche("X-Tudo", 18.50)
```

Tudo funciona bem — até alguém fazer algo inesperado.

```
1 xtudo = Lanche("X-Tudo", 18.50)
2 xtudo.preco = -999      # nenhum erro!
3 xtudo.preco = "gratis" # nenhum erro!
4 print(xtudo.preco)     # "gratis"
```

## Problema

Qualquer código externo pode colocar o objeto em um estado que não faz sentido. Um preço negativo ou textual viola as regras do nosso domínio.

Como proteger os dados internos de um objeto?

# Encapsulamento

Pense na caixa registradora do Podrão:

- O **visor** mostra o total (acesso público)
- A **gaveta** só abre com a chave do operador (acesso restrito)
- Ninguém coloca a mão diretamente no dinheiro

## Encapsulamento

Esconder os detalhes internos de um objeto e expor apenas uma interface controlada.

Python **não tem** modificadores de acesso como Java. Usa-se **convenções de nomenclatura**:

| Nome   | Convenção | Significado                      |
|--------|-----------|----------------------------------|
| nome   | público   | Acesso livre                     |
| _nome  | protegido | “Use com cuidado”                |
| __nome | privado   | Name mangling (dificulta acesso) |

## Filosofia Python

“We’re all consenting adults here” — a linguagem confia no programador, mas oferece mecanismos para sinalizar intenção.



Atributos com `__` (duplo sublinhado) sofrem **name mangling**:

```
1 class CaixaRegistradora:
2     def __init__(self):
3         self.__saldo = 0.0
4
5     def receber_pagamento(self, valor):
6         if valor > 0:
7             self.__saldo += valor
8
9     def ver_saldo(self):
10        return self.__saldo
```

```
1 caixa = CaixaRegistradora()
2 caixa.receber_pagamento(25.00)
3 print(caixa.ver_saldo()) # 25.0
4
5 print(caixa.__saldo)
6 # AttributeError: 'CaixaRegistradora' object
7 #   has no attribute '__saldo'
```

## Name mangling por dentro

O Python renomeia `__saldo` para `_CaixaRegistradora__saldo`.

```
1 # Funciona, mas NUNCA faça isso!
2 print(caixa._CaixaRegistradora__saldo) # 25.0
```

**@property — Acesso Controlado**

Em Java, usamos `getPreco()` e `setPreco()`. Em Python, isso fica verboso e pouco natural:

```
1  # Estilo Java em Python (evite!)
2  lanche.get_preco()
3  lanche.set_preco(20.0)
4
5  # Estilo Pythonico (queremos isto)
6  lanche.preco      # leitura
7  lanche.preco = 20.0 # escrita com validacao
```

A solução: `@property`.

```
1 class Lanche:
2     def __init__(self, nome, preco):
3         self._nome = nome
4         self.preco = preco # usa o setter!
5
6     @property
7     def preco(self):
8         return self._preco
9
10    @preco.setter
11    def preco(self, valor):
12        if valor < 0:
13            raise ValueError("Preco negativo!")
14        if valor > 500:
15            raise ValueError("Preco acima do limite!")
16        self._preco = valor
```

```
1 xtudo = Lanche("X-Tudo", 18.50)
2 print(xtudo.preco) # 18.5 (chama o getter)
3
4 xtudo.preco = 22.0 # OK (chama o setter)
5 xtudo.preco = -10  # ValueError: Preço negativo!
6 xtudo.preco = 1000 # ValueError: Preço acima do limite!
```

## Vantagem

O código externo usa sintaxe de atributo, mas internamente há validação. Mudança transparente.

Use @property **sem** definir o setter:

```
1 class Pedido:
2     def __init__(self):
3         self._itens = []
4
5     def adicionar(self, lanche, qtd=1):
6         self._itens.append((lanche, qtd))
7
8     @property
9     def total(self):
10         return sum(l.preco * q for l, q in self._itens)
```

```
1 pedido = Pedido()
2 pedido.adicionar(xtudo, 2)
3 print(pedido.total)    # 37.0
4 pedido.total = 0       # AttributeError!
```

## Cuidado!

Não use o mesmo nome para a property e o atributo interno.

```
1 class Errado:
2     @property
3     def preco(self):
4         return self.preco # chama a si mesmo!
5
6     @preco.setter
7     def preco(self, valor):
8         self.preco = valor # chama a si mesmo!
9         # RecursionError!
```

**Solução:** armazene em `self._preco` (com prefixo).



# Métodos Especiais (Dunders)

Métodos com duplo sublinhado: `__nome__`

São chamados automaticamente pelo Python em situações específicas:

| <b>Você escreve</b>         | <b>Python chama</b>             |
|-----------------------------|---------------------------------|
| <code>print(obj)</code>     | <code>obj.__str__()</code>      |
| <code>repr(obj)</code>      | <code>obj.__repr__()</code>     |
| <code>obj1 == obj2</code>   | <code>obj1.__eq__(obj2)</code>  |
| <code>obj1 &lt; obj2</code> | <code>obj1.__lt__(obj2)</code>  |
| <code>len(obj)</code>       | <code>obj.__len__()</code>      |
| <code>hash(obj)</code>      | <code>obj.__hash__()</code>     |
| <code>obj1 + obj2</code>    | <code>obj1.__add__(obj2)</code> |

```
1 class Lanche:
2     def __init__(self, nome, preco):
3         self._nome = nome
4         self._preco = preco
5
6 xtudo = Lanche("X-Tudo", 18.5)
7 print(xtudo)
8 # <__main__.Lanche object at 0x7f3b2c> (feio!)
```

Agora com dunders:

```
1     def __str__(self):
2         return f"{self._nome} - R${self._preco:.2f}"
3
4     def __repr__(self):
5         return f"Lanche('{self._nome}', {self._preco})"
```

```
1 print(xtudo)          # X-Tudo - R$18.50 (amigavel)
```

Por padrão, dois objetos com os mesmos dados são **diferentes**:

```
1 a = Lanche("X-Tudo", 18.5)
2 b = Lanche("X-Tudo", 18.5)
3 print(a == b) # False (compara identidade!)
```

Solução:

```
1 def __eq__(self, other):
2     if not isinstance(other, Lanche):
3         return NotImplemented
4     return (self._nome == other._nome
5           and self._preco == other._preco)
```

```
1 print(a == b) # True
```

```
1 def __lt__(self, other):
2     if not isinstance(other, Lanche):
3         return NotImplemented
4     return self._preco < other._preco
```

```
1 cardapio = [
2     Lanche("X-Tudo", 18.5),
3     Lanche("Misto", 8.0),
4     Lanche("Burgao", 22.0),
5 ]
6 for l in sorted(cardapio):
7     print(l)
8     # Misto - R$8.00
9     # X-Tudo - R$18.50
10    # Burgao - R$22.00
```

Ao definir `__eq__`, o Python desabilita `__hash__`:

```
1 lanches = {Lanche("X-Tudo", 18.5)}  
2 # TypeError: unhashable type: 'Lanche'
```

Solução:

```
1 def __hash__(self):  
2     return hash((self._nome, self._preco))
```

## Regra de ouro

Se `a == b`, então `hash(a) == hash(b)`. Use os mesmos atributos em ambos os métodos.

```
1 class Pedido:
2     def __init__(self):
3         self._itens = []
4
5     def adicionar(self, lanche, qtd=1):
6         self._itens.append((lanche, qtd))
7
8     def __len__(self):
9         return sum(q for _, q in self._itens)
```

```
1 pedido = Pedido()
2 pedido.adicionar(Lanche("X-Tudo", 18.5), 2)
3 pedido.adicionar(Lanche("Misto", 8.0), 1)
4 print(len(pedido)) # 3
```

| Método                | Operação                                  | Quando usar                      |
|-----------------------|-------------------------------------------|----------------------------------|
| <code>__init__</code> | Construtor                                | Sempre                           |
| <code>__str__</code>  | <code>print()</code> , <code>str()</code> | Saída amigável                   |
| <code>__repr__</code> | <code>repr()</code> , terminal            | Debug, logs                      |
| <code>__eq__</code>   | <code>==</code>                           | Comparação por valor             |
| <code>__lt__</code>   | <code>&lt;</code> , <code>sorted()</code> | Ordenação                        |
| <code>__hash__</code> | <code>set</code> , <code>dict</code>      | Após definir <code>__eq__</code> |
| <code>__len__</code>  | <code>len()</code>                        | Coleções                         |
| <code>__add__</code>  | <code>+</code>                            | Soma/concatenação                |
| <code>__sub__</code>  | <code>-</code>                            | Subtração                        |
| <code>__mul__</code>  | <code>*</code>                            | Multiplicação                    |



# Prática — Classe Fracao

Vamos construir uma classe completa que usa **tudo** que aprendemos:

- Validação no construtor (denominador  $\neq 0$ )
- Simplificação automática via MDC
- Properties somente-leitura
- `__str__`, `__repr__`
- `__eq__`, `__lt__`, `__hash__`
- `__add__`, `__sub__`, `__mul__`

```
1 from math import gcd
2
3 class Fracao:
4     def __init__(self, num, den=1):
5         if den == 0:
6             raise ValueError("Denominador nao pode ser zero!")
7         # sinal sempre no numerador
8         if den < 0:
9             num, den = -num, -den
10        divisor = gcd(abs(num), den)
11        self._num = num // divisor
12        self._den = den // divisor
13
14        @property
15        def num(self):
16            return self._num
17
18        @property
```

```
1 def __str__(self):
2     if self._den == 1:
3         return str(self._num)
4     return f"{self._num}/{self._den}"
5
6 def __repr__(self):
7     return f"Fracao({self._num}, {self._den})"
```

```
1 print(Fracao(4, 6)) # 2/3
2 print(Fracao(6, 3)) # 2
3 repr(Fracao(4, 6)) # Fracao(2, 3)
```

```
1  def __eq__(self, other):
2      if not isinstance(other, Fracao):
3          return NotImplemented
4      return (self._num == other._num
5              and self._den == other._den)
6
7  def __lt__(self, other):
8      if not isinstance(other, Fracao):
9          return NotImplemented
10     return self._num * other._den < other._num * self._den
11
12  def __hash__(self):
13     return hash((self._num, self._den))
```

```
1  def __add__(self, other):
2      if not isinstance(other, Fracao):
3          return NotImplemented
4      return Fracao(self._num * other._den
5                     + other._num * self._den,
6                     self._den * other._den)
7
8  def __sub__(self, other):
9      if not isinstance(other, Fracao):
10         return NotImplemented
11     return Fracao(self._num * other._den
12                  - other._num * self._den,
13                  self._den * other._den)
14
15  def __mul__(self, other):
16      if not isinstance(other, Fracao):
17          return NotImplemented
18      return Fracao(self._num * other._num,
```

```
1 a = Fracao(1, 2)
2 b = Fracao(1, 3)
3
4 print(a + b)          # 5/6
5 print(a - b)          # 1/6
6 print(a * b)          # 1/6
7 print(a == Fracao(2, 4)) # True (simplificado!)
```

```
1 fracs = [Fracao(3,4), Fracao(1,2), Fracao(2,3)]
2 print(sorted(fracs)) # [1/2, 2/3, 3/4]
```

```
1 # set() funciona porque temos __eq__ e __hash__
2 s = {Fracao(1,2), Fracao(2,4), Fracao(3,6)}
3 print(s)          # {1/2}
```

# Resumo



| Conceito              | Ferramenta Python                                                  |
|-----------------------|--------------------------------------------------------------------|
| Esconder dados        | <code>__atributo</code> (name mangling)                            |
| Acesso controlado     | <code>@property</code> / <code>@x.setter</code>                    |
| Representação textual | <code>__str__</code> / <code>__repr__</code>                       |
| Igualdade por valor   | <code>__eq__</code> + <code>__hash__</code>                        |
| Ordenação             | <code>__lt__</code>                                                |
| Operadores            | <code>__add__</code> , <code>__sub__</code> , <code>__mul__</code> |

## Cuidado!

1. Property com mesmo nome do atributo interno  $\Rightarrow$  recursão infinita
2. Definir `__eq__` sem `__hash__`  $\Rightarrow$  objetos não entram em `set/dict`
3. Acessar `_CaixaRegistradora__saldo` diretamente  $\Rightarrow$  quebra encapsulamento
4. Esquecer `isinstance` check nos dunders  $\Rightarrow$  erros com tipos mistos

## Aula 3 — Preview

- Herança: `class XTudo(Lanche)`
- Polimorfismo: mesmo método, comportamentos diferentes
- Composição vs. herança: quando usar cada um
- O food truck cresce: cardápio com subclasses

Dúvidas?